NeuroCore fNIRS/EEG Processing Pipelines

Documentazione tecnica completa estratta dal notebook Jupyter — Analisi dati SNIRF, metriche AUC, Resilienza e Transizioni

Cella 1 [Code - In Execution #11] — **NeuroCore SNIRF Recursive Reader v1.1**

```
# ============================================================ # NeuroCore SNIRF Recursive Reader v1.1 # Cerca e carica tutti i file SNIRF nelle sottocartelle # ============================================================ import mne import os # Path principale data_path = "/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/" # Funzione per trovare tutti i file .snirf ricorsivamente snirf_files = [] for root, dirs, files in os.walk(data_path): for f in files: if f.endswith('.snirf'): snirf_files.append(os.path.join(root, f)) print(f"Trovati {len(snirf_files)} file SNIRF:") for f in snirf_files: print(f) # Carica e visualizza il primo file trovato if snirf_files: fname = snirf_files[0] raw = mne.io.read_raw_snirf(fname, preload=True) # Info generali print("\n=== Info file ===") print(raw.info) print("\n=== Canali ===") print(raw.ch_names) # Plot interattivo dei dati raw.plot(scalings='auto', title="fNIRS SNIRF Data", show=True) else: print("Nessun file SNIRF trovato nelle sottocartelle.")
```

OUTPUT & TRACEBACK:

```
Trovati 22 file SNIRF:
/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/sub-VP01/nirs/._sub-VP01_task-pain_nirs.snirf
/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/sub-VP01/nirs/sub-VP01_task-pain_nirs.snirf ... [22 file elencati] Loading /Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/sub-VP01/nirs/._sub-VP01_task-pain_nirs.snirf OSError: Unable to synchronously open file (file signature not found)
Traceback: Cell In[11], line 26 raw = mne.io.read_raw_snirf(fname, preload=True) File ~/miniforge3/envs/neurocore/lib/python3.12/site-packages/h5py/_hl/files.py:566, in File.__init__ fid = make_fid(name, mode, userblock_size, fapl, fcpl, swmr=swmr)
```

Cella 2 [Code - In Execution #12] — **Filtro e Validazione File SNIRF**

```
import mne import os data_path = "/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/" snirf_files = [] for root, dirs, files in os.walk(data_path): for f in files: if f.endswith('.snirf'): snirf_files.append(os.path.join(root, f)) print(f"Trovati {len(snirf_files)} file SNIRF totali.") # Lista per file validi valid_files = [] for f in snirf_files: try: raw = mne.io.read_raw_snirf(f, preload=True) valid_files.append(f) print(f"✅ File valido: {f}") except OSError: print(f"❌ File non valido o corrotto: {f}") print(f"\nTotale file validi: {len(valid_files)}")
```

OUTPUT:

```
Trovati 22 file SNIRF totali. ❌ File non valido o corrotto: .../._sub-VP01_task-pain_nirs.snirf ✅ File valido: .../sub-VP01_task-pain_nirs.snirf ... [Filtrazione dei file nascosti macOS] Totale file validi: 10
```

Cella 3 [Code - In Execution #15] — **NeuroCore SNIRF Processing v3.0 - Precision Metrics**

```
# ============================================================ # NeuroCore SNIRF Processing v3.0 - Precision Metrics # Carica file validi, calcola AUC, resilienza funzionale e transizioni # ============================================================ import mne import os import numpy as np import pandas as pd import matplotlib.pyplot as plt from scipy.signal import butter, filtfilt # ------------------------------ # Configurazione path # ------------------------------ data_path = "/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/" output_path = os.path.join(data_path, "processed") os.makedirs(output_path, exist_ok=True) # ------------------------------ # Trova tutti i file SNIRF validi # ------------------------------ snirf_files = [] for root, dirs, files in os.walk(data_path): for f in files: if f.endswith('.snirf'): snirf_files.append(os.path.join(root, f)) valid_files = [] for f in snirf_files: try: raw = mne.io.read_raw_snirf(f, preload=True) valid_files.append(f) except OSError: pass # ------------------------------ # Funzioni metriche NeuroCore # ------------------------------ def bandpass_filter(signal, sfreq, low=0.01, high=0.2): b, a = butter(2, [low, high], btype='band', fs=sfreq) return filtfilt(b, a, signal) def calc_auc(signal, times): """Area sotto la curva (trapezoidale)""" return np.trapz(signal, x=times) def resilience_index(signal, sfreq, window_sec=5, threshold=0.01): """Indice di resilienza funzionale basato su rolling std""" window = int(window_sec * sfreq) rolling_std = pd.Series(signal).rolling(window, min_periods=1).std() stable = rolling_std < threshold transitions = stable.astype(int).diff().abs().sum() return stable.sum() / (1 + transitions) def transition_count(signal, sfreq, window_sec=5, threshold=0.01): """Conta transizioni significative tra stati stabili e instabili""" window = int(window_sec * sfreq) rolling_std = pd.Series(signal).rolling(window, min_periods=1).std() stable = rolling_std < threshold transitions = stable.astype(int).diff().abs().sum() return transitions # ------------------------------ # Elaborazione dati # ------------------------------ all_results = [] for f in valid_files: raw = mne.io.read_raw_snirf(f, preload=True) times = raw.times sfreq = raw.info['sfreq'] data = raw.get_data() # shape: canali x tempo for ch_idx, ch_name in enumerate(raw.ch_names): signal = data[ch_idx, :] signal_filt = bandpass_filter(signal, sfreq) auc = calc_auc(signal_filt, times) resilience = resilience_index(signal_filt, sfreq) transitions = transition_count(signal_filt, sfreq) all_results.append({ "file": os.path.basename(f), "channel": ch_name, "AUC": auc, "Resilience_Index": resilience, "Transitions": transitions }) df = pd.DataFrame(all_results) csv_file = os.path.join(output_path, "NeuroCore_fNIRS_metrics_precise.csv") df.to_csv(csv_file, index=False)
```

OUTPUT & TRACEBACK:

```
AttributeError: module 'numpy' has no attribute 'trapz' Traceback: Cell In[15], line 87, in auc = calc_auc(signal_filt, times) Cell In[15], line 51, in calc_auc return np.trapz(signal, x=times)
```

Cella 4 [Code - In Execution #17] — **Installazione Dipendenza Seaborn**

```
!pip install seaborn
```

OUTPUT:

```
       Collecting seaborn Downloading seaborn-0.13.2-py3-none-any.whl (294 kB) Successfully
installed seaborn-0.13.2
```

Cella 5 [Code - In Execution #4] **NeuroCore SNIRF Complete Report v5.1 (Versione Finale Stabile)**

```
# ============================================================ # NeuroCore SNIRF Complete Report v5.1 # Pipeline unica: AUC, Resilienza, Transizioni, PDF + CSV # Anti-errore trapz e installazione automatica seaborn # ============================================================ import mne import os import sys import subprocess import numpy as np import pandas as pd import matplotlib.pyplot as plt from scipy.signal import butter, filtfilt from matplotlib.backends.backend_pdf import PdfPages # ------------------------------ # Controllo automatico Seaborn # ------------------------------ try: import seaborn as sns except ImportError: print("Seaborn non trovato. Installazione in corso...") subprocess.check_call([sys.executable, "-m", "pip", "install", "seaborn"]) import seaborn as sns print("✅ Seaborn installato correttamente.") # ---------- # Configurazione path # ------------------------------ data_path = "/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/" output_path = os.path.join(data_path, "processed") os.makedirs(output_path, exist_ok=True) pdf_file = os.path.join(output_path, "NeuroCore_Report_v5_1.pdf") # ------------------------------ # Trova file SNIRF validi # ------------------------------ snirf_files = [] for root, dirs, files in os.walk(data_path): for f in files: if f.endswith('.snirf'): snirf_files.append(os.path.join(root, f)) valid_files = [] for f in snirf_files: try: raw = mne.io.read_raw_snirf(f, preload=True) valid_files.append(f) except OSError: continue print(f"Trovati {len(valid_files)} file validi.\n") # ------------------------------ # Funzioni metriche # ------------------------------ def bandpass_filter(signal, sfreq, low=0.01, high=0.2): b, a = butter(2, [low, high], btype='band', fs=sfreq) return filtfilt(b, a, signal) def calc_auc(signal, times): """AUC senza np.trapz, integrazione trapezoidale manuale""" dt = np.diff(times) y = signal[:-1] y_next = signal[1:] auc = np.sum((y + y_next) / 2 * dt) return auc def resilience_index(signal, sfreq, window_sec=5, threshold=0.01): window = int(window_sec * sfreq) rolling_std = pd.Series(signal).rolling(window, min_periods=1).std() stable = rolling_std < threshold transitions = stable.astype(int).diff().abs().sum() return stable.sum() / (1 + transitions) def transition_count(signal, sfreq, window_sec=5, threshold=0.01): window = int(window_sec * sfreq) rolling_std = pd.Series(signal).rolling(window, min_periods=1).std() stable = rolling_std < threshold transitions = stable.astype(int).diff().abs().sum() return transitions # ------------------------------ # Elaborazione dati + PDF # ------------------------------ all_results = [] with PdfPages(pdf_file) as pdf: for f in valid_files: raw = mne.io.read_raw_snirf(f, preload=True) times = raw.times sfreq = raw.info['sfreq'] data = raw.get_data() resilience_vals = [] channel_names = [] for ch_idx, ch_name in enumerate(raw.ch_names): signal = data[ch_idx, :] signal_filt = bandpass_filter(signal, sfreq) auc = calc_auc(signal_filt, times) resilience = resilience_index(signal_filt, sfreq) transitions = transition_count(signal_filt, sfreq) all_results.append({ "file": os.path.basename(f), "channel": ch_name, "AUC": auc, "Resilience_Index": resilience, "Transitions": transitions }) resilience_vals.append(resilience) channel_names.append(ch_name) # Plot segnale per canale plt.figure(figsize=(10,3)) plt.plot(times, signal_filt, label=f"{ch_name}") plt.title(f"{os.path.basename(f)} - {ch_name}") plt.xlabel("Tempo (s)") plt.ylabel("Segnale fNIRS filtrato") plt.legend() plt.tight_layout() pdf.savefig() plt.close() # Heatmap resilienza per file plt.figure(figsize=(10,3)) sns.heatmap([resilience_vals], annot=True, xticklabels=channel_names, cmap="viridis") plt.title(f"Heatmap Resilienza: {os.path.basename(f)}") plt.ylabel("Resilienza Funzionale") pdf.savefig() plt.close() # ------------------------------ # Distribuzioni aggregate # ------------------------------ df = pd.DataFrame(all_results) plt.figure(figsize=(10,4)) sns.histplot(df['AUC'], kde=True, color='blue', bins=20) plt.title("Distribuzione AUC") pdf.savefig() plt.close() plt.figure(figsize=(10,4)) sns.histplot(df['Resilience_Index'], kde=True, color='green', bins=20) plt.title("Distribuzione Resilienza Funzionale") pdf.savefig() plt.close() plt.figure(figsize=(10,4)) sns.histplot(df['Transitions'], kde=True, color='red', bins=20) plt.title("Distribuzione Transizioni Funzionali") pdf.savefig() plt.close() # ------------------------------ # Salvataggio CSV finale # ------------------------------ csv_file = os.path.join(output_path, "NeuroCore_fNIRS_metrics_v5_1.csv") df.to_csv(csv_file, index=False) print(f"\n✅ CSV finale salvato: {csv_file}") print(f"✅ Report PDF completo salvato: {pdf_file}")
```

OUTPUT ESECUZIONE FINALE:

```
      Trovati 10 file validi. Loading /.../sub-VP01_task-pain_nirs.snirf ... sub-VP13_task-
pain_nirs.snirf ✅ CSV finale salvato:
/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/processed/NeuroCore_fNIRS_
metrics_v5_1.csv ✅ Report PDF completo salvato:
/Volumes/KINGSTON/NeuroCore/neurocore_lab/data/EEG/Hypoalgesia_ds006902/processed/NeuroCore_Report
_v5_1.pdf
```